When the Envelope Suffices: Pre-Registered Falsification of Uncertainty Hardening in a Runtime Assurance Monitor for Autonomous Satellites

Sami Bey

Ingénieur HES-SO — Haute École Spécialisée de Suisse occidentale
Epalinges, Vaud, Switzerland — sami.bey@pedt-technologies.com
Reference implementation and campaign data: github.com/Stab123/runtime-assurance-monitor (MIT)

Working paper — September 2026

Version 4 — September 2026. Changes from v3: the 205.8 s τ_violation is explicitly tied to the monitor's nominal model (the Monte Carlo plants' physical τ_violation, 100.9–185.6 s, are covered by a new sensitivity analysis in Section 5.4 — the plant-level ratio r_plant exceeds 1 for part of the runs without changing the result); the scope of the compile-time wall is conditioned on the declared nominal model and action bounds; an eighth defect (configuration dt_s ignored by the campaign layer, with zero numerical impact) is documented in Section 6. Archived code, data and history: DOI 10.5281/zenodo.22552147.

Abstract

We report the design, implementation, and pre-registered falsification campaign of a runtime assurance monitor for autonomous satellites. The monitor supervises an unverifiable decision layer: at every decision cycle it emits exactly one verdict — authorize, minimally modify, fall back, or declare the situation undetermined — and records a self-describing decision trace. Its modification semantics specialize Wabersich and Zeilinger's predictive safety filter to scalar actions, with a fixed-iteration bisection and a deterministic, statically checkable fallback policy; its scenario and cadence follow the CySat-I in-flight runtime verification experiment. The engineering bet under test (hypothesis H4) is that first-order uncertainty hardening — pessimistic trajectory evaluation at $x \mp k\sigma \cdot \sigma$ and fallback when the estimated uncertainty exceeds a per-constraint threshold — buys safety robustness at an acceptable operational cost. Instead of tuning the mechanism until it looks good, we froze a falsification criterion and a versioned configuration, then ran a Monte Carlo campaign (four arms, common random numbers, innovation-based uncertainty estimation that never reads the prediction model's parameters, plant parameters drawn independently and deliberately unfavorably to the monitor). The envelope-only arm records zero observed safety violations on all runs of the tested campaigns; every level at which the uncertainty threshold actually fires costs strictly more fallback and strictly less mission delivery than the envelope-only arm, with no observed safety benefit. A fourth arm isolates the cost of pessimistic evaluation from the cost of the uncertainty threshold. We state the negative result with its exact scope — including the structural reason the envelope suffices here. A second campaign varies the single structural lever we identified — the fallback arming delay, measured by the ratio $r = \tau_{arming}/\tau_{violation}$ (formal $\tau_{violation}$: 205.8 s on the nominal model) — from 0.58, its P2 value, up to 0.92: the envelope alone remains at zero violation across the whole compilable band, and beyond $r \approx 0.95$ the constraint set is rejected at compile time — under the declared nominal monitor model and action bounds, the regime in which hardening could justify itself is not deployable; the Monte Carlo plants' physical ratio, which exceeds 1 for part of the runs, does not change this result. We finally report the eight defects caught while the unit test suite was fully green — four interaction defects before the first campaign, then three code defects and one documentation defect — and the unchanged-configuration re-run protocol that followed.

Keywords: *runtime assurance — predictive safety filter — runtime verification — autonomous satellites — falsification — Monte Carlo campaign — common random numbers — decision traceability*

1. Introduction

Autonomous satellites increasingly embark decision layers — planners, learning-based controllers, resource optimizers — whose behavior cannot be certified by classical means. The runtime assurance (RTA) answer to this problem is architectural: let the complex layer propose, and place a simple, verifiable component in the path of its actions, able to authorize, modify, or block them [2][3]. The present work sits in the frame of an onboard runtime assurance monitor specified for decisional autonomy under space computation constraints (internal specification RAM-SPEC-0001 [8]), motivated by the "Runtime-Assured Autonomous Satellite" axis identified in the S4 programme context (ESA Phi-Lab Switzerland / ESDI) [11].

Two lines of work anchor the design. From the predictive safety filter of Wabersich and Zeilinger [4][5], the monitor takes the semantics of *minimal modification*: a candidate action that would lead to a constraint violation is replaced by the closest action that keeps a safe backup trajectory available, and only infeasibility of that problem triggers a fallback. From the CySat-I experiment of Aurandt, Jones and Rozier [1] — the first in-flight demonstration of runtime verification triggering autonomous fault recovery on a CubeSat — it takes the methodology: constraints derived from natural-language requirements, a 5-second monitoring cadence, deliberate fault injection, and recovery by predefined strategies.

The monitor we built hardens the predictive evaluation with first-order uncertainty: predicted trajectories are evaluated at the unfavorable bound $x \mp k\sigma \cdot \sigma$ of the estimated state, and an excessive estimated uncertainty forces fallback regardless of the nominal estimate (hypothesis H4). H4 is an engineering bet: it should buy robustness against model–world discrepancy at an acceptable cost in fallback rate and mission delivery. Bets of this kind are usually "validated" by showing scenarios in which they help. We chose the opposite discipline, closer to Popper than to tuning: we froze a falsification criterion and a versioned configuration *before* running the campaign, drew plant parameters from ranges deliberately unfavorable to the monitor, and committed to keeping the result whatever it was.

The result is negative, and clean. An envelope-only monitor (no uncertainty hardening) achieves zero safety violations on every run of the campaign; every activation level of H4 costs more fallback and less mission delivery, without any safety benefit to show for it. Because the falsification criterion was fixed in advance and the control arm sits at its ceiling — zero violation is the best possible outcome — no threshold adjustment can rescue the hypothesis within this regime.

Contributions. (i) A reference implementation of the monitor semantics — four verdicts, scalar specialization of the predictive safety filter, compile-time validation of fallback reachability, self-describing decision trace — with requirement-level traceability to the specification. (ii) A falsification methodology for assurance mechanisms: pre-registered criteria, common random numbers with a per-run witness, innovation-based uncertainty estimation that avoids circularity, independent and unfavorable plant draws, and bit-reproducible artifacts (versioned configurations, code hashes embedded in the results). (iii) A correctly scoped negative result on first-order uncertainty hardening, including a fourth experimental arm that separates the cost of pessimistic evaluation from the cost of the uncertainty threshold, a structural criterion delimiting the regime in which the result holds (Section 7), and a report of the eight defects that unit tests did not see — four interaction defects before the first campaign, four more (three in code, one in documentation) in September 2026, followed by a full unchanged-configuration re-run.

What this paper does not claim. It does not show that uncertainty hardening is useless in general. It shows that *in a regime where the safety envelope alone already suffices, first-order hardening cannot justify its cost* — a weaker and truer statement, and the one the experiment was actually designed to test.

2. Background and related work

2.1 Runtime assurance

The Simplex architecture introduced the pattern of pairing a high-performance, unverified controller with a high-assurance, verified one, plus a switching logic that hands control to the safe side when the plant approaches the boundary of the recoverable region [2]; Sha's formulation — "use simplicity to control complexity" — separates critical requirements, kept by simple software, from desirable properties, delegated to complex software [3]. Classical Simplex *switches* to the backup controller. The monitor studied here instead *filters*: its primary intervention is a modified action that stays as close as possible to the candidate, with fallback reserved for infeasibility — which is precisely the semantics of the predictive safety filter.

2.2 Predictive safety filter

Wabersich and Zeilinger certify learning-based controllers by solving, at each step, a constrained finite-horizon problem that searches for a safe backup trajectory toward a safe set, modifying the proposed input only as much as necessary [5]; the linear case appeared in [4]. Section 3.2 recalls the formulation and states our three deliberate deviations: a non-optimized but statically checkable backup policy, a scalar specialization replacing the quadratic program by a fixed-iteration bisection, and a pessimistic evaluation at a declared (not calibrated) confidence level.

2.3 Runtime verification in flight: CySat-I

Aurandt, Jones and Rozier flew the R2U2 engine on the CySat-I CubeSat's on-board computer: 22 mission-time LTL specifications elicited from natural-language electrical power system (EPS) requirements, evaluated by a 5-second FreeRTOS task, with eight emulated EPS fault scenarios triggering autonomous recoveries — and a genuine firmware bug discovered by the monitor during ground testing [1]. CySat-I verified the *state* of the satellite against temporal properties. The monitor presented here verifies the *actions* of an autonomous decision layer against safety constraints — a different object, the same discipline: natural-language requirements compiled into checkable form, a fixed cadence, injected faults, and predefined recovery. Our demonstration scenario (Section 3.4) deliberately mirrors their EPS setting.

2.4 Internal documents

The work is governed by three unpublished working documents cited throughout: the functional specification RAM-SPEC-0001 [8], the trace-budget computation note RAM-NOTE-0001 [9] (which sizes the decision trace at 16/48/160 bytes per cycle and justifies event-based extended retention by the rarity of non-nominal verdicts), and an ECSS positioning note RAM-ECSS-0001 [10], which maps the monitor's requirements onto ECSS-E-ST-70-11C Rev.1 [6] and ECSS-E-ST-70-41C [7]. A state-of-the-art review on onboard runtime assurance [11] provides the programme context. None of the results reported below depends on the content of these internal documents: the reference implementation, the frozen configurations and the campaign results are public and self-contained.

3. The monitor (P0)

3.1 Verdict semantics and interfaces

The monitor is synchronous with the decision loop (5 s period, following CySat-I's cadence). At every cycle it emits *exactly one* verdict (requirement RA-FUN-001): AUTORISE — the candidate action is transmitted unchanged; MODIFIE — the closest admissible action is transmitted instead; REPLI — the deterministic fallback policy takes over; INDETERMINE — inputs are invalid or stale; execution treats it as fallback, but the trace keeps it distinct (RA-FUN-002). The decision layer's only input to the monitor is the candidate action; the trace stores a 3-byte hash of it, never internal state (RA-IND-001). A missing candidate at the deadline is itself a verdict trigger: fallback, cause TIMEOUT_ACTION (RA-IND-003). Fallback is latched: return to nominal requires an explicit, counted and timestamped criterion — N consecutive fully safe cycles plus a minimum dwell time (RA-FUN-006). Every verdict is notified to the platform FDIR (interface IF-4), and the constraint set can only be updated under explicit authorization, its version number appearing in every subsequent trace record (IF-6, RA-IND-004).

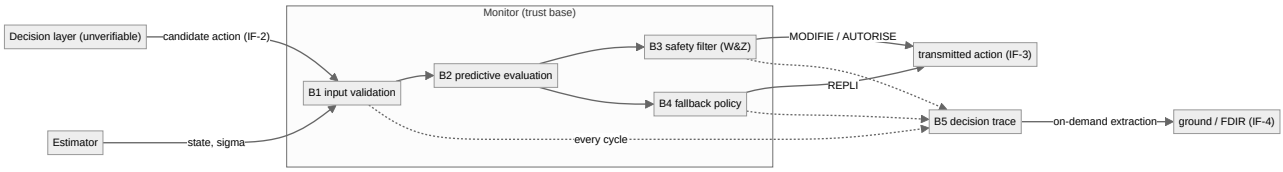

Figure 1. Monitor architecture (blocks B1–B5 of RAM-SPEC-0001). The decision layer is a black box; only its candidate actions cross the interface. The trace recorder is never on the critical path: its saturation degrades the record, never the verdict.

3.2 Scalar specialization of the predictive safety filter

Wabersich and Zeilinger's filter solves, at each decision instant, the problem

$$\min \text{ over } v_0,\dots,v_{N-1} \text{ of } \|v_0 - u_{cand}\|^2 \quad \text{s.t.} \quad z_{k+1} = f(z_k, v_k),\ z_0 = x(t),\ z_k \in X,\ v_k \in U,\ z_N \in X_f \tag{1}$$

and applies v_0; infeasibility hands control to a backup trajectory. Our specialization keeps the semantics and replaces the machinery, in three documented steps. **(a) Backup = fallback policy.** The backup trajectory is not optimized: it is the unrolled deterministic fallback policy (payload off), which is statically checkable — more conservative than the optimized backup, deliberately, because this policy belongs to the trust base. **(b) Scalar action.** For a scalar action with a monotonicity property (higher payload current is never safer), the admissible set of v_0 is an interval [u_min, û]; the upper bound is found by bisection in a *fixed* number of iterations (32), giving a statically bounded worst-case execution profile in the spirit of the specification's resource requirements. The verdict MODIFIE transmits û — the minimal modification of the candidate in the sense of (1). **(c) Pessimistic evaluation.** Each constraint is checked at the unfavorable bound $x \mp k\sigma \cdot \sigma$ of the estimated state, with $k\sigma = 3.0$ corresponding to a *declared* confidence level p_S ≈ 99.7 % under Gaussian assumptions — declared, not calibrated: connecting this level to the actual estimator's properties is future work, and until then every margin reads “at declared p_S”.

3.3 Compile-time validation

Two requirements are discharged when a constraint set is compiled, not in flight. The prediction horizon covers the maximal fallback arming delay plus a margin (RA-FUN-004) — a fallback that takes 120 s to arm is only creditworthy after those 120 s in the evaluated trajectory, during which the currently transmitted action persists. And the fallback must actually be reachable: from the guard boundary of every constraint, the trajectory with the *most unfavorable admissible action persisted during arming* (both bounds are tested) must not cross any raw safety threshold (RA-FUN-005). An invalid set is rejected at compile time with `JeuInvalide`. The guard margin is the switching backoff that buys the fallback time to act — it is not the safety limit, and the compile-time check is deliberately run against the raw thresholds. Concretely, on the demonstration model: a 120 s arming delay passes (worst-case state of charge at the end of arming: 0.3583 for a 0.35 threshold — recomputed after the September 2026 correction, under which the persisted action was applied one step too long), a 300 s delay is rejected — the exact wall sits at 196 s (Section 5.4) — and the same constraint would have been accepted had the check ignored the arming delay.

3.4 Demonstration scenario and decision trace

The demonstration mirrors CySat-I's EPS setting: natural-language requirements (“battery charge must never fall below the undervoltage threshold”, “battery temperature must not exceed 45 °C”) compiled into two scalar constraints with guard margins, arming delays and per-constraint uncertainty thresholds; a two-and-a-half-orbit scenario with an aggressive, deliberately infeasible payload demand and three injected faults — an out-of-bounds action drift, a degraded estimator, and a frozen decision layer. Figure 2 shows the monitor's behavior: the out-of-bounds candidate is clamped to the admissible bound; the estimator degradation triggers fallback on uncertainty grounds alone; the frozen decision layer triggers fallback on timeout; and the battery state of charge rides the guard threshold without ever crossing the safety limit.

One feature of this figure is worth flagging before the campaign, because it is the point the campaign overturns. The uncertainty-driven fallback visible during the estimator degradation *looks like* the mechanism earning its place. Section 5 shows that in this regime it is redundant: the envelope alone catches the same cases without it. The demonstration establishes that the mechanism fires as specified; it does not establish that its firing prevents anything.

Every cycle is recorded (RA-TRC-001) in a fixed-format, self-describing record — three variants of 16, 48 and 160 bytes as sized by the trace-budget note [9] — in a fixed-capacity ring buffer. Records carry the constraint-set version, the verdict, a cause that distinguishes envelope-driven from uncertainty-driven fallback (RA-TRC-004), the determining constraint and its normalized margin, hashes of the candidate and transmitted actions, and a CRC; a record can be decoded autonomously for blind audit, without access to the decision layer's internals (RA-TRC-002, RA-TRC-005). A non-nominal verdict freezes a context window in mission, voiding the rarity economics on which the trace budget rests [9]. Saturation degrades the record, never the verdict (RA-RES-004), and every lost record is counted (RA-RES-005). A heartbeat per cycle is checked by an independent watchdog (RA-SUR-002).

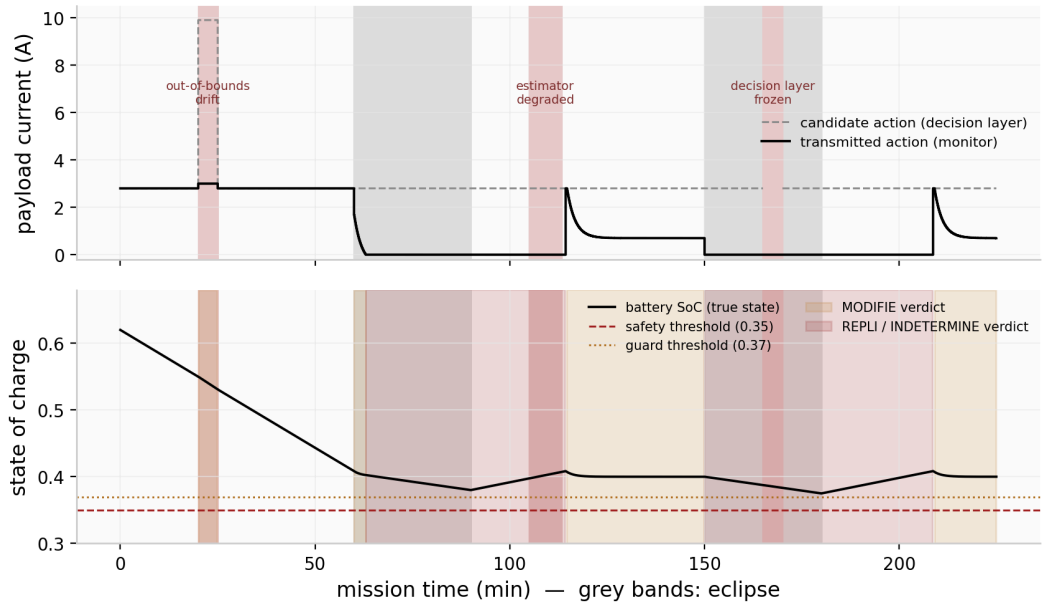

Figure 2. Demonstration scenario (2.5 orbits, 5 s cycles). Top: candidate action proposed by the decision layer (dashed) and action actually transmitted by the monitor (solid); red bands mark the three injected faults, grey bands the eclipses. Bottom: true battery state of charge against the safety threshold (0.35) and the guard threshold (0.37); amber and red shading mark MODIFIE and REPLI/INDETERMINE verdicts. The demand profile is deliberately infeasible — the scenario is a stress test, not an operations plan; operational acceptability is the subject of Section 5.

3.5 Verification status

The reference implementation (Python, ~1 100 lines with tests) ships with 23 unit tests, each attached to a requirement. Table 1 summarizes the traceability. P0 is not flight code: memory bounds and worst-case execution time are only prepared (fixed-iteration bisection, fixed-capacity buffers), the prediction model is perfect, and the action is scalar; these debts are stated in the implementation's README and in Section 7.

Table 1. Requirement-to-test traceability (condensed; requirement identifiers refer to RAM-SPEC-0001 [8]).

Requirement	Mechanism	Test
RA-FUN-001/002	exactly one verdict per cycle; INDETERMINE traced distinctly, executed as fallback	`test_ra_fun_001/002`
RA-FUN-003	per-constraint uncertainty threshold forces fallback regardless of the nominal estimate	`test_ra_fun_003`
RA-FUN-004/005	horizon covers arming delay; fallback reachability checked at compile time against raw thresholds, worst-case action persisted during arming	`test_ra_fun_004`, `test_ra_fun_005` (×2)
RA-FUN-006	latched fallback; explicit, counted, timestamped return criterion	`test_ra_fun_006`
RA-IND-001/003/004	candidate action only; hash in trace; missing candidate → fallback; authorized, versioned constraint-set update	`test_ra_ind_*`
RA-RES-004/005	trace saturation never blocks a verdict; losses counted	`test_ra_res_004_005`
RA-TRC-001..007	one record per cycle; self-describing; distinct causes; versioned schema; frozen window on fallback only; ring buffer wrap without loss	`test_paquet_c_*`, `test_tampon_reboucle_sans_perte`, …
RA-SUR-002	per-cycle heartbeat + independent watchdog	`test_ra_sur_002`

4. Falsification methodology

4.1 The hypothesis and the pre-registered criterion

Hypothesis H4 holds that first-order uncertainty hardening — pessimistic evaluation at $x \mp k\sigma \cdot \sigma$ plus fallback on excessive uncertainty — buys safety robustness at acceptable operational cost.

H4 combines two distinct mechanisms, and the campaign is designed to tell them apart. The first is pessimistic evaluation at kσ = 3 (Section 3.2c); the second is the per-constraint uncertainty threshold, which is requirement RA-FUN-003. Arm D isolates the cost of the first, and the D-to-C gap isolates the cost of the second. Falsifying H4 therefore falsifies the conjunction, and the arm decomposition attributes the cost to each component separately — a distinction the reference implementation's README follows as well.

> ***Falsification criterion (frozen, config P2.1).*** *H4 is supported only if there exists an uncertainty-threshold grid point whose safety-violation rate is strictly lower than the envelope-only arm's, at comparable fallback and delivery cost. If the envelope-only arm is at zero violation, no grid point can satisfy the criterion, and H4 is falsified in this regime.*

Three operational metrics are recorded per run: the **violation rate** (fraction of cycles in which the *true* plant state crosses a *raw* safety threshold), the **fallback rate** (fraction of cycles with verdict REPLI or INDETERMINE), and **mission delivery** (energy actually delivered to the payload over energy requested during payload windows). Confidence intervals are 95 % Student-t intervals over per-run rates; proportions of runs use Wilson score intervals.

4.2 Arms and common random numbers

Four arms share every draw. **A**: no monitor — the (clamped) candidate drives the plant directly. **B**: envelope only — the monitor with kσ = 0 and an infinite uncertainty threshold. **C**: full H4 — kσ = 3 and the uncertainty threshold set to a grid point. **D** (added in the second campaign version, see 5.2): pessimistic evaluation only — kσ = 3, infinite threshold. Comparability rests on common random numbers: for run *i*, plant parameters are drawn once from `seed_plant + i`, and measurement noise is re-generated identically in every arm from `seed_noise + i`, with exactly two Gaussian draws per cycle in every arm (the estimator also runs in arm A, unused). The two base seeds are separated by more than the number of draws any run consumes, so the plant and noise streams cannot overlap. Only the monitor's configuration differs. A per-run witness — the raw noise deviate stored before any arithmetic — is compared across arms; it matched in 32/32 runs of the first two campaigns and 300/300 of the third. Two earlier witness designs failed honestly and are worth reporting: witnessing the *estimated* state fails because estimates diverge across arms by design (that divergence is the object of the experiment), and reconstructing the deviate by floating-point subtraction is not bit-identical across arms.

4.3 Uncertainty estimation without circularity

The critical point of the design is where σ comes from. If σ derived from the same information the monitor uses to predict, the reasoning would be circular and the campaign would measure nothing. The estimator therefore computes σ from *measurement residuals only*. With z the measurement, $\bar{x}$ the one-step prediction under the nominal model, and the innovation $\nu = z - \bar{x}$, the estimator applies a proportional correction and maintains exponential statistics of the innovation:

$$\sigma_i = \sqrt{\mathrm{EWMA}_\beta(\nu_i^2)} + \left|\mathrm{EWMA}_\beta(\nu_i)\right| \times H \qquad (2)$$

where H is the monitor's prediction horizon in steps. The first term covers measurement noise and instantaneous error; the second propagates the estimated systematic bias — e.g. a slow model–world drift such as a real battery capacity below the nominal one — over the horizon. Both terms carry the units of the measured state, and the absolute value keeps σ non-negative under a signed bias. The only information shared with the monitor is the nominal propagation model, which is unavoidable: innovations are computed against the very model the monitor predicts with, so σ measures exactly the monitor's prediction error. σ never reads the plant's parameters — the model–world gap reaches it exclusively through measurements. At the 5 s cadence the per-cycle parametric drift sits below the unit measurement noise; it only becomes visible through the averaged bias, which is what the second term exists for.

4.4 Plant parameterization — deliberately unfavorable

For each run, the true plant's parameters are drawn uniformly (the least informative choice — no invented tails) within the bounds of Table 2, while the monitor keeps the nominal P0 model. The draw is deliberately unfavorable: the monitor assumes a 10 Ah battery and a 0.5 A bus, the plant draws 5–9 Ah and 0.45–0.6 A — the monitor is systematically optimistic about capacity, and every run measures exactly the model–world discrepancy H4 was supposed to absorb. A degradation window multiplies the measurement noise by six for 45 minutes of each 3.75-hour run.

Table 2. Plant parameter draws (uniform) versus the monitor's nominal model.

Parameter	Lower	Upper	Monitor nominal
Battery capacity (Ah)	5.0	9.0	10.0
Charge current in sunlight (A)	1.0	1.4	1.2
Bus consumption (A)	0.45	0.60	0.50
Heating coefficient (°C/(A·s))	1.2×10^{-3}	2.2×10^{-3}	1.5×10^{-3}
Thermal relaxation (1/s)	3×10^{-4}	7×10^{-4}	5×10^{-4}
Initial state of charge	0.50	0.65	—
Initial temperature (°C)	15	25	—

4.5 Why the operational thresholds are not arbitrated

The versioned configuration carries three operational acceptance values (a ceiling on violation rate, a ceiling on fallback rate, a floor on mission delivery) explicitly marked *not arbitrated*. This is a deliberate position, and we state it because it is unattackable: any number written today, after seeing the results, would be a number chosen with knowledge of the outcome — a reviewer would see it, and would be right to see it. The falsification conclusion does not depend on those values: it rests on a comparison between arms — arm B reaches zero violation, nothing can do better, and every grid point of arm C costs more than B on both other metrics. The thresholds would only serve a different question — "is this system operationally acceptable?" — which this work cannot answer alone, because it requires a satellite operator. We see this as the right hook for a discussion with a lab or an industrial partner: here are the curves; where would you place the floor? Table 4 provides a concrete illustration: arm B's delivery stands at 90.81 % with a confidence-interval lower bound at 89.95 % — a floor set at 90 % would be missed by 0.05 point on the conservative reading, and passed on the central one. Choosing the reading is already part of the acceptance decision; it does not belong to this work.

4.6 Reproducibility and pre-registration

Every campaign version is a frozen JSON configuration (seeds, plant bounds, grid, thresholds) whose modification history is append-only: P2.1 (falsification), P2.2 (fourth arm and extended grid), P2.3 (statistical strengthening). Results files embed the SHA-256 hashes of the campaign and monitor code. Because every arm re-seeds its own noise, later versions reproduce earlier ones bit for bit on their common runs — verified explicitly: the parallel runner (P2.3) reproduces the serial P2.2 results bit for bit, and P2.2's runs 0–31 are reproduced inside P2.3. The public repository separates proof from replay: the `empreintes/` directory keeps the exact bytes that produced the results (verification only), `ram_p0/` and `ram_p2/` provide the executable copy with relative paths, and the `verifier_empreintes.py` script prints, for each campaign, the expected and computed hashes module by module; a continuous-integration workflow reruns the test suite and this verification on every push.

Hashes establish that the reported results were produced by the configuration and code shipped with them; they do not, by themselves, establish that the falsification criterion predates the campaign, since a repository history can be rewritten by its author. That trust horizon is now covered by two independent, non-rewritable archives: Zenodo, which timestamps every published version of the repository (concept DOI 10.5281/zenodo.22552147, one DOI per version), and Software Heritage, which preserves the full git graph — the configuration-before-results order is readable there for P2.1, P2.2, P2.3 and P3.1. The version tags are annotated but not GPG-signed: timestamping and integrity rest on these two archives. Readers who wish to verify the pre-registration claim should rely on these deposits rather than on the author's word.

5. Results

5.1 Campaign P2.1 — the falsification

The first campaign (32 runs per grid point, 2.5 orbits and 2 700 cycles per run) is reported in Table 3 exactly as it ran. The unmonitored arm A violates safety in 8 of 32 runs (5.36 % of cycles on average). The envelope-only arm B violates nothing — zero violation on every run. Since the pre-registered criterion required a grid point *strictly* below B, and B sits at zero, no grid point can qualify: H4 is falsified. And the cost side makes the falsification actionable rather than technical: B buys its zero at 7.03 % fallback and 88.1 % delivery, while the best grid point of C pays 10.25 % fallback and 83.5 % delivery — more cost, no benefit, on all three metrics at once.

Table 3. Campaign P2.1 (32 runs per point, 2 700 cycles per run): the falsification. Arm B — envelope only — violates nothing on any run; every grid point of arm C (full H4) costs more fallback and less delivery. Rates are means over runs with 95 % Student-t intervals; run proportions use Wilson score intervals.

Arm	σ thr.	N	violation rate % [CI95]	runs w/ viol. [Wilson]	fallback rate % [CI95]	delivery % [CI95]
A	—	32	5.36 [1.18, 9.54]	8/32 [0.133, 0.421]	0.00 [0.00, 0.00]	100.00 [100.00, 100.00]
B	—	32	0.00 [0.00, 0.00]	0/32 [0.000, 0.107]	7.03 [4.84, 9.23]	88.11 [84.26, 91.95]
C	0.010	32	0.00 [0.00, 0.00]	0/32 [0.000, 0.107]	39.20 [33.75, 44.66]	49.85 [42.44, 57.26]
C	0.020	32	0.00 [0.00, 0.00]	0/32 [0.000, 0.107]	23.22 [21.47, 24.97]	72.21 [68.97, 75.44]
C	0.030	32	0.00 [0.00, 0.00]	0/32 [0.000, 0.107]	18.69 [16.83, 20.55]	79.50 [75.81, 83.19]
C	0.050	32	0.00 [0.00, 0.00]	0/32 [0.000, 0.107]	10.36 [7.71, 13.01]	83.52 [78.92, 88.12]
C	0.080	32	0.00 [0.00, 0.00]	0/32 [0.000, 0.107]	10.25 [7.58, 12.92]	83.52 [78.92, 88.12]

The violation rate of arm A falls from 5.36 % here to 2.53 % at N = 300 (Table 4). The two values are consistent — the P2.1 interval [1.18, 9.54] contains the later estimate — but the reader comparing the two tables should read the N = 32 figure as the noisy one, not as evidence of a changed scenario: seeds, bounds and margins are identical.

5.2 Campaign P2.2 — correcting the experiment, not the conclusion

Two defects of the P2.1 experimental plan were identified in review. First, arm C varied two things at once: it activated the uncertainty threshold *and* raised kσ from 0 to 3, so the gap between B and C confounded the cost of pessimistic evaluation with the cost of the threshold. Second, the grid saturated: estimated σ rarely exceeded 0.05, so the top of the grid was already "H4 off" — the 0.05 and 0.08 points were bit-identical. P2.2 corrected both, and only those: a fourth arm D (kσ = 3, infinite threshold) isolates the pessimism cost along the chain B → D → C, and the grid was extended downwards into the region where σ actually lives. Seeds, plant bounds, margins and run count were left untouched; P2.2 reproduces P2.1 bit for bit on every common point.

Two findings resulted. Mechanically, arm D is identical to C@0.08 in every run — direct evidence that the threshold never fires there, so the entire B-to-C@0.08 gap is pessimism cost. Quantitatively, pessimistic evaluation alone (B → D) costs, at N = 32, +3.3 points of fallback and −4.6 points of delivery (the N = 300 estimate of Table 4 gives +2.9 and −5.0 points); the threshold (D → C) adds up to +84 points of fallback at the finest grid point, where the system spends 94.6 % of cycles in fallback and delivers 4.7 % of requested energy. Neither correction rescued H4; both sharpened its falsification. These were corrections of the experimental plan — the scenario, bounds and margins were never retuned, and both campaigns are preserved in full.

5.3 Campaign P2.3 — statistical strengthening

The falsification criterion was already decided at N = 32, but the zero-violation claim of arm B was only bounded at 10.7 % (Wilson upper bound with 0/32). P2.3 re-runs the frozen configuration with N = 300 — runs 0–31 reproduce P2.2 bit for bit — using a run-parallel executor validated against the serial campaign. Arm B records zero violations in 300 runs (Wilson 95 % upper bound: 1.3 %, down from 10.7 % at N = 32), and so do arm D and every grid point of arm C. Table 4 reports the full grid — post-correction re-run figures (September 2026), differing by at most 0.04 point from the values published in v2; Figure 3 compares the four arms on both cost metrics (arm C is shown at the σ = 0.03 threshold; Table 4 gives the full grid).

Table 4. Campaign P2.3 (300 runs per point; runs 0–31 reproduce P2.2 bit for bit). Arm D isolates pessimistic evaluation (kσ = 3, infinite threshold); the gap from D to a C row is the cost of the uncertainty threshold itself. Post-correction re-run figures (September 2026) — they differ by at most 0.04 point from the v2 values.

Arm	σ thr.	N	violation rate % [CI95]	runs w/ viol. [Wilson]	fallback rate % [CI95]	delivery % [CI95]
A	—	300	2.53 [1.82, 3.24]	68/300 [0.183, 0.277]	0.00 [0.00, 0.00]	100.00 [100.00, 100.00]
B	—	300	0.00 [0.00, 0.00]	0/300 [0.000, 0.013]	6.57 [6.01, 7.14]	90.81 [89.95, 91.66]
D	∞	300	0.00 [0.00, 0.00]	0/300 [0.000, 0.013]	9.43 [8.73, 10.12]	85.83 [84.72, 86.93]
C	0.005	300	0.00 [0.00, 0.00]	0/300 [0.000, 0.013]	94.89 [94.38, 95.40]	4.54 [3.90, 5.19]
C	0.008	300	0.00 [0.00, 0.00]	0/300 [0.000, 0.013]	59.10 [56.99, 61.21]	31.91 [29.65, 34.17]
C	0.010	300	0.00 [0.00, 0.00]	0/300 [0.000, 0.013]	40.20 [38.53, 41.87]	48.00 [45.86, 50.14]
C	0.015	300	0.00 [0.00, 0.00]	0/300 [0.000, 0.013]	24.98 [24.45, 25.51]	67.34 [66.04, 68.64]
C	0.020	300	0.00 [0.00, 0.00]	0/300 [0.000, 0.013]	22.63 [22.14, 23.12]	74.13 [73.32, 74.95]
C	0.030	300	0.00 [0.00, 0.00]	0/300 [0.000, 0.013]	18.11 [17.59, 18.63]	81.52 [80.70, 82.33]
C	0.050	300	0.00 [0.00, 0.00]	0/300 [0.000, 0.013]	9.60 [8.91, 10.29]	85.82 [84.71, 86.93]
C	0.080	300	0.00 [0.00, 0.00]	0/300 [0.000, 0.013]	9.43 [8.73, 10.12]	85.83 [84.72, 86.93]

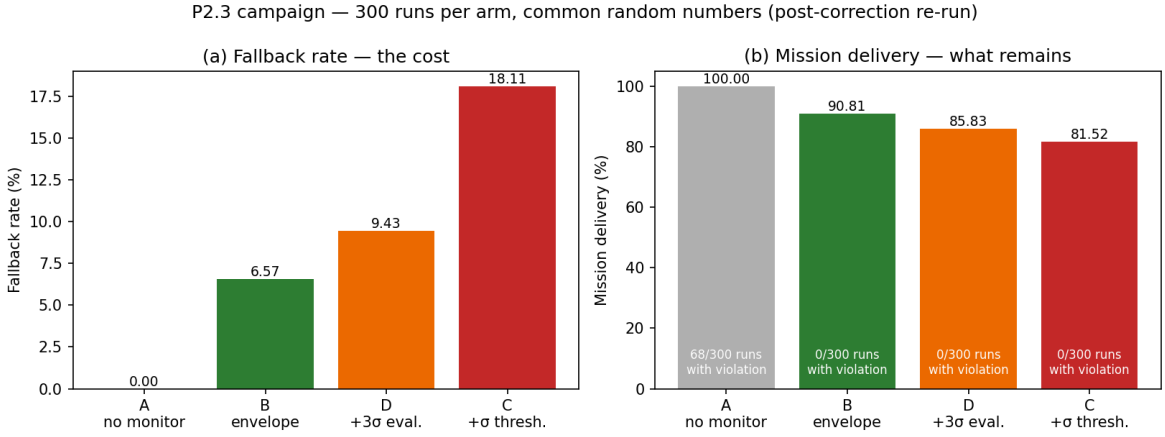

Figure 3. Campaign P2.3 (300 runs per arm, post-correction re-run; figure regenerated by `figures/faire_figures.py` from the published campaign file). Left: fallback rate per arm. Right: mission delivery — the annotation reports runs with violations (arm A: 68/300; arms B, D and C: 0/300, and zero is the ceiling). Arm C is shown at the σ = 0.03 threshold; Table 4 gives the full grid.

5.4 Campaign P3.1 — pushing the r ratio toward the marginal regime

Section 7 conditions the usefulness of hardening on a regime where fallback authority is marginal relative to time-to-violation, measured by the ratio r = τ_arming/τ_violation. Two τ_violation must be distinguished. The **nominal τ_violation** is measured on the monitor's nominal model — C_BATT = 10 Ah, I_BASE = 0.5 A, most unfavorable action u = 3 A persisted, guard margin 0.02, eclipse —: raw threshold crossed at 205.8 s. It sets the scale of the **r_nominal** ratio: in P2, τ_arming = 120 s, i.e. r_nominal = 0.58 — the fallback has nearly twice the time it needs. This is **not** the physical time-to-violation of the campaign plants: those draw C_BATT ∈ [5; 9] Ah and I_BASE ∈ [0.45; 0.60] A, hence a shorter τ_violation,plant, and a physical ratio r_plant = τ_arming/τ_violation,plant that can exceed 1 (sensitivity analysis below). Campaign P3 pushes the single lever τ_arming toward r = 1, everything else frozen (seeds, plant bounds, margins, σ grid, cycles — configuration committed before execution).

The marginal regime turned out to be **undeployable within the declared framework** — and this statement is conditional. The power pilot (arm B only, N = 30, run before the criterion was frozen) first located a wall: compile-time validation (RA-FUN-005, Section 3.3) accepts the constraint set up to and including τ_arming = 195 s (residual margin of +0.0002 on state of charge) and rejects it from 196 s — i.e. r_nominal ≈ 0.95. This wall depends on the nominal model (C_BATT = 10 Ah), on the declared action bounds (u_max = 3 A), on the safety margins and on the constraint set: **under the declared nominal monitor model and action bounds, compile-time validation rejects constraint sets beyond the identified arming-delay boundary** — it is not a general physical impossibility. P3.1 then documents the compilable band with statistical power: three points τ_arming ∈ {140; 170; 190} s, N = 300, four arms, unchanged σ

grid, with a frozen criterion in two clauses — the campaign is informative only if arm B violates somewhere, and H4 is supported only if some (r, σ threshold) pair does strictly better than B on violations at comparable costs.

Result: **no violations were observed in 300 Monte Carlo runs**, at each of the three points, for arms B, D and C; the 95 % Wilson upper bound for the run-level violation probability is approximately 1.3 %. Arm A — independent of r by construction — reproduces P2.3 bit for bit at every point (68/300 runs with violations), as the protocol's non-regression required. The first clause of the criterion being unmet, **P3.1 is inconclusive for H4**. Cost, on the other hand, grows with τ_arming: arm B's fallback rises from 10.18 % to 15.60 % then 18.77 %, and its delivery falls from 89.05 % to 86.47 % then 84.91 % (Table 6, Figure 4). (Historical configuration: the frozen grid indexed r on a reference constant τ_violation = 400 s — hence the labels 0.35 / 0.425 / 0.475; the actually executed delays are 140, 170 and 190 s. The values reported here, 0.68 / 0.83 / 0.92 against the measured nominal τ_violation of 205.8 s, are the **corrected r_nominal**, not to be confused with the campaign's historical labels.)

Sensitivity: the plants' physical r. The compile-time wall, like the r_nominal values above, is indexed on the nominal model; the campaign plants can violate earlier. A retrospective analysis (`ram_p3/analyse_sensibilite_r.py`, results in `ram_p3/resultats_sensibilite_r.json`) computes each run's τ_violation,plant from its actual parameters, regenerated through the frozen configuration's deterministic draw — same convention as the nominal one: guard boundary, u = 3 A, eclipse. No campaign is re-run, no data is modified. The τ_violation,plant span 100.9 to 185.6 s: already in P2 (τ_arming = 120 s), 50 runs out of 300 have r_plant > 1.05; at the τ_arming = 190 s point, 292 runs out of 300 do (r_plant up to 1.88). Across all these subpopulations — including the top r_plant decile, where arm A violates in 14 runs out of 30 — arm B remains at zero observed violation, and the B < D < C cost ordering is unchanged. Where hardening physically had its best chance, it brought no observed safety benefit and cost availability: the negative result does not rest on the nominal model alone.

Table 6. Campaign P3.1 (300 runs per point, post-correction re-run): costs against the arming delay. B, D and C are at zero observed violation everywhere; arm A, independent of r by construction, reproduces P2.3 bit for bit at every point (2.53 %; 68/300 runs) and is not repeated. Arm C is shown at the σ = 0.03 threshold. The r column is the corrected r_nominal (measured nominal τ_violation, 205.8 s); the campaign's historical labels, indexed on the 400 s reference constant, are 0.35 / 0.425 / 0.475.

τ_arming (s)	r	Arm	fallback rate % [CI95]	delivery % [CI95]
140	0.68	B	10.18 [9.40, 10.96]	89.05 [88.24, 89.86]
140	0.68	D	12.85 [12.00, 13.71]	83.94 [82.88, 84.99]
140	0.68	C	20.81 [20.10, 21.53]	79.53 [78.76, 80.29]
170	0.83	B	15.60 [14.64, 16.56]	86.47 [85.71, 87.23]
170	0.83	D	17.86 [16.85, 18.86]	81.46 [80.46, 82.47]
170	0.83	C	24.31 [23.38, 25.23]	77.04 [76.28, 77.79]
190	0.92	B	18.77 [17.71, 19.83]	84.91 [84.19, 85.63]
190	0.92	D	20.86 [19.77, 21.95]	79.86 [78.88, 80.85]
190	0.92	C	26.30 [25.26, 27.34]	75.40 [74.61, 76.18]

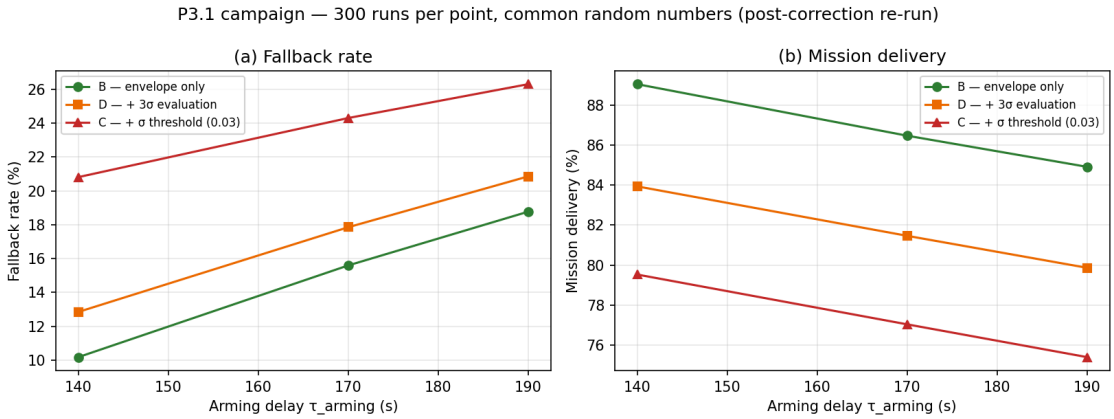

Figure 4. Campaign P3.1 (300 runs per point, post-correction re-run; figure regenerated by `figures/faire_figures.py`). Fallback rate (left) and mission delivery (right) against the fallback arming delay. The cost of every monitored arm grows with τ_arming, and the B < D < C ordering holds at every point. Beyond 195 s (r ≈ 0.95), the constraint set is rejected at compile time.

6. Lessons learned: what the unit tests did not see

The monitor's test suite — twenty tests at the time, twenty-three since the regression tests added in September 2026 — was fully green when adversarial review found four defects, all of them in *interactions* between components rather than inside any component.

One of the four deserves to be stated outside a table, because it bears directly on the credibility of the result. Defect 4 — the bisection's inner call silently dropping the `k_sigma` argument — would have corrupted any arm running a non-default kσ, and that means arm B (kσ = 0), the control arm on which the entire falsification rests. Had it survived into the campaign, the zero-violation baseline would have been an artifact. It did not: the SHA-256 hashes of the monitor modules embedded in the P2.1, P2.2 and P2.3 results files are identical, and they correspond to the corrected code; only the campaign driver differs between versions. (The P2.1 driver did not survive the execution environment — it was edited in place when moving to four arms; its hash, preserved in the P2.1 results file, remains verifiable, and the four-arm version published in `empreintes/` reproduces its arms A, B and C run by run.) No reported number was produced by those four defects, fixed before the first campaign — and the hashes, recomputable by `verifier_empreintes.py` on the bytes in `empreintes/`, let a third party check that statement without trusting the author. That sentence does not, however, cover the four defects found later (Table 5, rows 5–8): see the re-run paragraph below.

Table 5. The eight defects found while the test suite was fully green. Defects 1–4 (interaction): adversarial review, fixed before the first campaign — no published number stems from them. Defects 5–8 (September 2026, after v2): three in code, one in documentation — this version's figures are those of the post-correction re-run (defect 8, with zero numerical impact, requires no re-run).

#	Defect	Why the tests missed it
1	Ring buffer: on a frozen slot, the write head did not advance — every record after a freeze was silently lost until extraction.	No test ever wrapped the buffer; the freeze path and the wrap path were each tested in isolation.
2	Context freeze triggered on every non-nominal verdict, including MODIFIE — the frozen window merged to cover the whole mission (2,032 freeze calls), contradicting the trace budget's rarity assumption.	The freeze mechanism was tested, its *economics* never — the defect is visible only over a full scenario.
3	Compile-time fallback validation passed an arming delay of zero to the trajectory check — a 300 s arming delay would have been accepted although the constraint is violated during arming.	The check existed and was tested — with parameters that made it vacuous. A dedicated test (300 s arming must raise `JeuInvalide`) now locks the behavior.
4	The bisection's inner call dropped the `k_sigma` argument, silently restoring the default — any arm with a non-default kσ (arm B, kσ = 0) would have been corrupted.	Invisible until an experiment varied the parameter; found while re-reading the code *for the campaign*, not by the suite.
5	Fallback persistence one step too long during arming (`k <= pas_armement`: τ + 5 s instead of τ) — a convention shared by compile time and runtime, hence no safety hole and no bias between arms, but a temporal label off by one step (RA-FUN-004).	The horizon tests shared the same wrong convention; found by cross-reading against the requirement. A regression test now locks the behavior.
6	Projection of a candidate below u_min: the filter returned the *upper* bound of the admissible interval instead of u_min — a non-minimal modification. Never reached in the campaigns (candidates ∈ {0; 3} A).	No test covered that side of the interval, and the campaign's candidate domain never goes there; tests added on both sides.
7	The README's σ table frozen on draft values, inconsistent with the campaign file — the published data were never affected.	No test re-reads the documentation; found during the re-run, while comparing the table to the file.
8	Campaign layer: the simulation read the `DT` constant from `demo_eps.py` instead of the configuration's declared `dt_s` — a configuration value ignored. Both equal 5.0 s in every frozen configuration: **zero numerical impact**. P2 code kept intact (frozen scientific artefact): defect documented, not fixed.	The configuration declared the very value being used — the defect was invisible to any numerical comparison; found in the September 2026 review (configuration → code traceability).

The September 2026 re-run. Defects 5 and 6 were found after v2 was published — and the published results had, this time, been produced by the affected code. Every campaign (P2.2, P2.3, P3 pilot, P3.1) was therefore re-run on continuous integration with **strictly unchanged** configurations, only the code being fixed; P2.1 was not re-run (historical pilot lost, plan already superseded by P2.2) and its pre-correction bytes are archived in `empreintes/pre_correction/`. Verdict: violation counts unchanged everywhere, fallback and delivery shifted by at most 0.04 point, compile-time wall moved by exactly one pilot grid point — as the fix predicted — and non-regressions re-verified on the new data. All qualitative conclusions hold; this version's figures are those of the re-run. Defect 8, found in the same review, requires no re-run: `dt_s` equals 5.0 s in every frozen configuration, exactly the constant being used — the executed values are identical, verified by bit-for-bit non-regression against the published results, and the P2 code is kept intact as a scientific artefact. Full protocol and before/after values: the repository's CHANGELOG.

Two *designs* of the CRN witness also failed honestly ahead of the campaigns (Section 4.2) — design iterations caught before any use, not counted among the eight defects. The pattern is consistent: unit tests validate components against their specification; these defects lived in composition — buffer policy × freeze policy, compile-time check × real arming delays, filter plumbing × experimental arms — and only adversarial review plus a pre-registered falsification protocol caught them. We read this as an argument for the methodology, not merely as an anecdote.

7. Scope and limitations

The control arm sits at its ceiling. In a regime where the envelope alone is already perfect, a hardening mechanism can only cost — the experiment has no headroom in which H4 could have shown a benefit. The falsification is therefore exactly as strong as the regime it covers: it does not demonstrate that uncertainty hardening is useless where the envelope is *not* sufficient (deeper model error, genuinely adversarial dynamics, longer horizons), only that deploying it where the envelope suffices buys nothing.

Why the envelope suffices here — and where it would not. The reason is structural, not incidental. The fallback policy — payload off — removes the dominant discharge term, so its authority over the constrained state is large compared with both the disturbance and the estimation error, and the guard margin buys it enough time to act before any raw threshold is crossed. Under those conditions no amount of uncertainty information can improve on an envelope that is never late: the plant draws a capacity down to half the nominal value, and the envelope still catches every case. The generalizable statement is therefore a design criterion rather than a scenario report: *first-order uncertainty hardening can only pay where fallback authority is marginal relative to time-to-violation*. That condition — not scenario difficulty in the abstract — defines the regime in which H4 deserves a fair test, and it is testable in advance of any campaign: it depends on the arming delay, the guard margin and the disturbance magnitude, not on the outcome.

The condition is measured, not merely stated. The scenario's formal $\tau_{violation}$ is 205.8 s — on the **monitor's nominal model** (C_{BATT} = 10 Ah, I_{BASE} = 0.5 A, u = 3 A, margin 0.02), not on the campaign plants ($\tau_{violation,plant}$ from 100.9 to 185.6 s, Section 5.4). The ratio $r_{nominal} = \tau_{arming}/\tau_{violation}$ is therefore 0.58 in P2. Campaign P3.1 (Section 5.4) pushed it to 0.68, 0.83 then 0.92 — no violation observed for the envelope alone (0/300 at every point, Wilson bound 1.3 %) — and beyond $r_{nominal} \approx 0.95$ (τ_{arming} = 196 s), compile-time validation rejects the constraint set: **under the declared nominal monitor model and action bounds**, the regime in which fallback authority becomes marginal is not a regime that is hard to survive, it is a regime RA-FUN-005 forbids to deploy — without this being a general physical impossibility. The per-plant sensitivity analysis (r_{plant} up to 1.88, Section 5.4) does not change the finding. Testing H4 fairly will therefore require a different fallback policy — one that does not remove the dominant disturbance term — not another tuning of the arming delay.

Statistical power. Zero observed violations in N runs is an upper bound, not a proof; P2.3 tightens the Wilson bound from 10.7 % (N = 32) to 1.3 % (N = 300). The cycle-level violation confidence intervals share the same limit. The per-run rate distributions are zero-inflated and right-skewed, so the Student-t intervals reported in Tables 3 and 4 should be read as indicative for arm A; the arm-to-arm comparisons that carry the conclusion do not depend on them.

The operational thresholds are not arbitrated (Section 4.5): the falsification does not need them, but the operational question — where the fallback ceiling and the delivery floor should sit — does, and it belongs to an operator.

Prototype debts. P0 assumes a perfect prediction model within the campaign's parametric family (model–world gap enters as parametric scatter and noise, not as structural error), a scalar action under a monotonicity assumption, constant σ over the horizon, and a declared-but-uncalibrated confidence level p_S. Fallback reachability is checked by boundary sampling, not formal reachability. Memory and WCET bounds are prepared (fixed-iteration bisection, fixed-capacity buffers) but only provable after the C port of blocks B2–B4.

8. Conclusion

A runtime assurance monitor for autonomous satellites was specified, implemented as a reference prototype with requirement-level traceability, and subjected to the discipline its own specification demanded: a falsification criterion frozen before the campaign, unfavorable plant draws, common random numbers with a verified witness, non-circular uncertainty estimation, and versioned configurations that preserve every campaign. In the tested regime, where the safety envelope already achieved zero observed violations, uncertainty hardening provided no additional observed safety benefit and reduced availability. Campaign P3.1 tightens the scope further: pushing the $r_{nominal}$ ratio from 0.58 to 0.92 does not revive H4 — including in the plant subpopulations where the physical ratio exceeds 1 — and the regime in which it would get its chance is rejected at compile time ($r_{nominal} \approx 0.95$) — undeployable under the declared nominal monitor model and action bounds.

The honest formulation is also the publishable one: the claim is scoped to its regime, the decomposition between pessimism cost and threshold cost is explicit, and the thresholds that would turn the curves into an operational acceptance decision are left, deliberately, to an operator. The natural continuations are the calibrated connection between $k\sigma$ and the estimator's true error distribution, the vector-action quadratic program of the full predictive safety filter, and formal fallback reachability. The continuation that matters most for H4 itself is a regime satisfying the condition stated in Section 7 — fallback authority marginal relative to time-to-violation. Lengthening the arming delay cannot get there: campaign P3.1 showed that, under the declared nominal monitor model and action bounds, compile-time validation rejects the constraint set beyond $r_{nominal} \approx 0.95$. What is needed is a fallback that does not remove the dominant disturbance term — a different policy, not a different tuning — where the same frozen-criterion methodology can give the hypothesis the fair chance it did not need here.

References

[1] Aurandt, A., Jones, P. H., & Rozier, K. Y. (2022). Runtime verification triggers real-time, autonomous fault recovery on the CySat-I. In J. V. Deshmukh, K. Havelund, & I. Perez (Eds.), *NASA Formal Methods (NFM 2022), Lecture Notes in Computer Science* (Vol. 13260, pp. 816–825). Springer. https://doi.org/10.1007/978-3-031-06773-0_45

[2] Seto, D., Krogh, B., Sha, L., & Chutinan, A. (1998). The Simplex architecture for safe online control system upgrades. In *Proceedings of the 1998 American Control Conference* (Vol. 6, pp. 3504–3508). IEEE.

[3] Sha, L. (2001). Using simplicity to control complexity. *IEEE Software*, 18(4), 20–28. https://doi.org/10.1109/MS.2001.936213

[4] Wabersich, K. P., & Zeilinger, M. N. (2018). Linear model predictive safety certification for learning-based control. In *2018 IEEE Conference on Decision and Control (CDC)* (pp. 7130–7135). IEEE.

[5] Wabersich, K. P., & Zeilinger, M. N. (2021). A predictive safety filter for learning-based control of constrained nonlinear dynamical systems. *Automatica*, 129, 109597. https://doi.org/10.1016/j.automatica.2021.109597

[6] ECSS-E-ST-70-11C Rev.1 (15 October 2025). *Space segment operability*. ECSS Secretariat, ESA-ESTEC.

[7] ECSS-E-ST-70-41C (15 April 2016). *Telemetry and telecommand packet utilization*. ECSS Secretariat, ESA-ESTEC.

[8] RAM-SPEC-0001 v0.1 (2026). *Moniteur d'assurance runtime embarqué — spécification fonctionnelle*. Internal working document, unpublished.

[9] RAM-NOTE-0001 (2026). *Budget de trace de décision — note de calcul*. Internal working document, unpublished.

[10] RAM-ECSS-0001 (2026). *Mapping ECSS du moniteur d'assurance*. Internal working document, unpublished.

[11] Bey, S. (2026). *Runtime assurance embarqué sous contrainte spatiale — état de l'art*. Internal working document, unpublished.